

Table of Contents

Chapter 4: Computational Thinking — Question Bank	2
Section A: Multiple Choice Questions (MCQs)	2
4.1 Computational Thinking	2
4.2 Principles of Computational Thinking	4
4.3 Algorithm Design Methods — Flowcharts	5
4.3 Algorithm Design Methods — Pseudocode	7
4.4 Evaluation Techniques for an Algorithm	9
4.5 Dry Run	9
4.6 LARP	11
4.7 Error Identification and Debugging	12
Section B: Short Questions	14
4.1 Computational Thinking	14
4.2 Principles of Computational Thinking	16
4.3 Algorithm Design Methods	16
4.4 Evaluation Techniques	18
4.5 Dry Run and Simulation	19
4.6 LARP	20
4.7 Error Identification and Debugging	21

Chapter 4: Computational Thinking — Question Bank

Section A: Multiple Choice Questions (MCQs)

Instructions: Choose the best answer. Correct answers are marked with .

4.1 Computational Thinking

Q1. What is Computational Thinking?

- A) A method of typing code quickly
 - B) A problem-solving process that involves skills and techniques to solve complex problems in a way executable by a computer
 - C) A way to design computer hardware
 - D) A technique used only by software engineers
-

Q2. Which of the following is NOT a key component of Computational Thinking?

- A) Decomposition
 - B) Pattern Recognition
 - C) Memorization
 - D) Abstraction
-

Q3. Decomposition in computational thinking means:

- A) Combining multiple solutions into one
 - B) Writing code in different programming languages
 - C) Breaking down a complex problem into smaller, manageable tasks
 - D) Finding patterns in data
-

Q4. Which of the following is the best example of decomposition?

- A) Recognizing that all squares follow the same area formula
 - B) Breaking "building a birdhouse" into steps like designing, gathering materials, cutting wood, and assembling
 - C) Hiding unnecessary details when making tea
 - D) Writing a step-by-step recipe
-

Q5. Pattern recognition in computational thinking involves:

- A) Breaking problems into sub-problems
 - B) Identifying and understanding regularities or patterns within a set of data or problems
 - C) Creating step-by-step instructions
 - D) Hiding complex details
-

Q6. The area of a square with side length 4 using the pattern of consecutive odd numbers is:

- A) $1 + 3 + 5 = 9$
 - B) $1 + 3 = 4$
 - C) $1 + 3 + 5 + 7 = 16$
 - D) $1 + 3 + 5 + 7 + 9 = 25$
-

Q7. Abstraction is best described as:

- A) Creating a detailed list of all problem components
 - B) Finding recurring patterns in data
 - C) Hiding complex details while exposing only the necessary parts
 - D) Dividing a problem into sub-problems
-

Q8. Which example best demonstrates abstraction?

- A) Listing all ingredients to bake a cake
 - B) Breaking a website into layout, content, and code
 - C) Describing "making a cup of tea" as just: boil water, add tea, wait, pour
 - D) Drawing arrows to show the flow of a process
-

Q9. An algorithm is defined as:

- A) A visual diagram using symbols and arrows
 - B) An informal description of a problem
 - C) A method to identify patterns in data
 - D) A precise, step-by-step set of instructions to solve a problem or complete a task
-

Q10. Which of the following is the correct first step in planting a tree algorithm from the chapter?

- A) Water the tree generously
 - B) Fill the hole with soil
 - C) Choose a suitable spot in your garden
 - D) Add mulch around the base
-

4.2 Principles of Computational Thinking

Q11. What is the FIRST and most important step in computational thinking?

- A) Solution Design
 - B) Pattern Recognition
 - C) Problem Understanding
 - D) Problem Simplification
-

Q12. According to Albert Einstein's quote referenced in the chapter, if he had an hour to solve a problem, he would spend how long thinking about the problem?

- A) 30 minutes
 - B) 45 minutes
 - C) 55 minutes
 - D) 50 minutes
-

Q13. Problem simplification involves:

- A) Selecting the fastest algorithm
 - B) Breaking a problem into smaller, more manageable sub-problems
 - C) Drawing a flowchart for the problem
 - D) Writing pseudocode for the problem
-

Q14. Solution selection and design requires:

- A) Only writing code
 - B) Decomposing the problem into sub-tasks
 - C) Evaluating different approaches and selecting the most efficient one, then creating a detailed plan
 - D) Only identifying patterns
-

Q15. To design a website, which is an example of problem simplification?

- A) Writing the entire website code at once
 - B) Testing the website with 100 users
 - C) Breaking tasks into designing the layout, creating content, and coding the functionality
 - D) Choosing between HTML and CSS
-

4.3 Algorithm Design Methods — Flowcharts

Q16. What are flowcharts?

- A) Written descriptions of algorithms using plain language
 - B) Visual representations of the steps in a process, depicted using different symbols connected by arrows
 - C) Programs that run algorithms automatically
 - D) Tables that compare algorithms
-

Q17. Which symbol is used to represent the START or END of a process in a flowchart?

- A) Rectangle
 - B) Diamond
 - C) Parallelogram
 - D) Oval (Terminal)
-

Q18. Which flowchart symbol represents a decision point?

- A) Rectangle
- B) Oval
- C) Diamond
- D) Parallelogram

Q19. A Rectangle in a flowchart represents:

- A) Input or output operation
 - B) Start or end of a process
 - C) A decision point
 - D) A process, task, or operation to be performed
-

Q20. Which symbol represents Input/Output in a flowchart?

- A) Oval
 - B) Diamond
 - C) Rectangle
 - D) Parallelogram
-

Q21. Arrows in a flowchart are used to:

- A) Represent decisions
 - B) Show the start of the process
 - C) Show the direction of flow and connect symbols to indicate the sequence of steps
 - D) Represent input/output
-

Q22. Flowcharts were popularized in the early days of computing by:

- A) Alan Turing and Bill Gates
 - B) John von Neumann and Herman Goldstine
 - C) Albert Einstein and Isaac Newton
 - D) Steve Jobs and Dennis Ritchie
-

Q23. In the flowchart for finding even or odd numbers, which operation is performed on the number?

- A) Division by 10
 - B) Modulus by 2 (remainder when divided by 2)
 - C) Multiplication by 2
 - D) Addition of 2
-

Q24. In the login system flowchart, what happens after 5 failed login attempts?

- A) The user is asked to reset password
 - B) The system restarts
 - C) The account is locked and the user is alerted
 - D) The system grants limited access
-

Q25. Which is NOT listed as an importance of flowcharts in the chapter?

- A) Clarity
 - B) Communication
 - C) Speed of execution
 - D) Documentation
-

4.3 Algorithm Design Methods — Pseudocode

Q26. Pseudocode is best described as:

- A) Actual programming code written in Python
 - B) A visual diagram using standard symbols
 - C) A method of representing an algorithm using simple and informal language combining programming structure with plain English readability
 - D) A compiled program that runs on a computer
-

Q27. Which of the following is TRUE about pseudocode?

- A) It can be directly run on a computer
 - B) It uses only mathematical symbols
 - C) It focuses on the logic of the algorithm without worrying about the syntax of a specific programming language
 - D) It is only used after writing the actual code
-

Q28. In the pseudocode for checking even or odd, which condition is used?

- A) `number / 2 == 0`
- B) `number + 2 == 0`
- C) `No. % 2 == 0` (modulus equals zero)
- D) `number * 2 == 0`

Q29. In the pseudocode for checking if a number is prime, what is returned if the number is less than or equal to 1?

- A) True
- B) "Prime"
- C) "Not checked"
- D) False

Q30. In the prime number pseudocode, the loop runs from 2 to:

- A) number / 2
- B) number - 1
- C) $\text{sqrt}(\text{number})$
- D) number + 1

Q31. The pseudocode for credential checking uses which variable to store the correct username?

- A) correctUser
- B) validUsername
- C) inputUser
- D) trueUsername

Q32. Which of the following is listed as a benefit of using pseudocode?

- A) It runs faster than actual code
- B) It replaces the need for flowcharts completely
- C) It generates executable programs automatically
- D) It is a universal way to convey algorithm steps, making it easier to discuss with others

Q33. How does pseudocode differ from a flowchart?

- A) Pseudocode uses symbols; flowcharts use plain language
- B) Pseudocode uses structured plain language read sequentially; flowcharts use graphical symbols and arrows to show flow visually
- C) Pseudocode is only for beginners; flowcharts are for experts
- D) There is no difference between them

4.4 Evaluation Techniques for an Algorithm

Q34. Time complexity measures:

- A) How much memory an algorithm uses
- B) How many lines of code are in an algorithm
- C) How fast or slow an algorithm performs and how running time changes as input size increases
- D) How many decisions an algorithm makes

Q35. Time complexity is usually expressed using:

- A) Percentage notation
- B) Roman numerals
- C) Big O notation (e.g., $O(n)$, $O(\log n)$, $O(n^2)$)
- D) Decimal notation

Q36. Space complexity measures:

- A) The number of steps in an algorithm
- B) How fast an algorithm runs
- C) The number of decisions in an algorithm
- D) The amount of memory an algorithm uses relative to input size

4.5 Dry Run

Q37. A dry run involves:

- A) Compiling the program on a computer
- B) Testing the program with 1000 data inputs automatically
- C) Manually going through the algorithm with sample data to identify any errors
- D) Drawing a flowchart for the algorithm

Q38. In the dry run of the "Sum of Two Numbers" flowchart with inputs 3 and 5, what is the output?

- A) 3
- B) 5

- C) 53
 - D) 8
-

Q39. In the dry run of the "FindMax" pseudocode with inputs 10 and 15, what is the output?

- A) 10
 - B) False
 - C) 15
 - D) 25
-

Q40. In the FindMax dry run, why is max assigned to num2 (15)?

- A) Because num2 is always larger
 - B) Because num1 is undefined
 - C) Because the condition $\text{num1} > \text{num2}$ ($10 > 15$) is False, so the else branch executes
 - D) Because the algorithm is incorrect
-

Q41. What is Simulation in the context of algorithms?

- A) Writing pseudocode for a real problem
 - B) Drawing a flowchart manually
 - C) Using computer programs to create a model of a real-world process to test ideas without trying them in real life
 - D) Performing a dry run of pseudocode
-

Q42. Which of the following is listed as a benefit of simulation?

- A) It increases algorithm complexity
 - B) It requires real-world experiments
 - C) It is cost-effective, safe, and repeatable
 - D) It eliminates the need for flowcharts
-

Q43. Which real-world application uses simulation as mentioned in the chapter?

- A) Baking a cake
- B) Planting a tree

- C) Weather forecasting by meteorologists
 - D) Designing a login system
-

4.6 LARP

Q44. LARP stands for:

- A) Language Algorithm Running Protocol
 - B) Logic of Algorithms for Resolution of Problems
 - C) Logical Array Resolution Program
 - D) Loop Algorithm Runtime Process
-

Q45. In LARP syntax, which command is used to display a message?

- A) PRINT
 - B) DISPLAY
 - C) OUTPUT
 - D) WRITE
-

Q46. In LARP syntax, which command is used to take input from a user?

- A) GET
 - B) INPUT
 - C) SCAN
 - D) READ
-

Q47. A LARP algorithm must always begin with which command?

- A) BEGIN
 - B) RUN
 - C) START
 - D) OPEN
-

Q48. A LARP algorithm must always end with which command?

- A) STOP
- B) CLOSE

- C) FINISH
 - D) END
-

Q49. In LARP's even/odd algorithm, what condition is checked?

- A) $\text{number} / 2 == 1$
 - B) $\text{number} \% 2 == 0$ (to check for even)
 - C) $\text{number} * 2 == 0$
 - D) $\text{number} - 2 == 0$
-

4.7 Error Identification and Debugging

Q50. Which type of error occurs when something is written incorrectly in the algorithm, such as missing a step or using the wrong symbol?

- A) Runtime Error
 - B) Logical Error
 - C) Execution Error
 - D) Syntax Error
-

Q51. A Runtime Error occurs when:

- A) The algorithm has wrong logic
 - B) A symbol is missing in the flowchart
 - C) The algorithm is being executed and an impossible operation is attempted, such as dividing by zero
 - D) The algorithm produces the wrong output silently
-

Q52. Which type of error is hardest to detect because the algorithm still runs but produces incorrect results?

- A) Syntax Error
 - B) Runtime Error
 - C) Logical Error
 - D) Compilation Error
-

Q53. Which type of error is easiest to find because LARP usually points it out?

- A) Logical Error
 - B) Runtime Error
 - C) Syntax Error
 - D) Semantic Error
-

Q54. What does the LARP error message "Undefined Variable" mean?

- A) A step is missing in the algorithm
 - B) You divided by zero
 - C) You are using a variable that hasn't been defined yet
 - D) The algorithm has a logical error
-

Q55. What does the LARP error message "Invalid Operation" indicate?

- A) A missing step in the algorithm
 - B) A variable used before definition
 - C) An incorrect condition in a decision
 - D) Performing an operation that is not allowed, such as dividing by zero
-

Q56. The term "debugging" originates from:

- A) A mathematical term for fixing logical errors
 - B) A programming language keyword
 - C) An actual moth (bug) found causing problems in an early computer
 - D) The name of a computer scientist
-

Q57. Which of the following is a common error message in LARP?

- A) "Loop Not Found"
 - B) "Flowchart Missing"
 - C) "Missing Step"
 - D) "Symbol Overflow"
-

Q58. Computational thinking can be applied in:

- A) Only computer science
 - B) Only mathematics
 - C) Only biology
 - D) Various fields including biology, mathematics, engineering, and daily life
-

Q59. Which of the following is the correct order of steps in the "birdhouse" decomposition example?

- A) Cut Wood → Gather Materials → Design → Assemble → Paint → Install
 - B) Design → Gather Materials → Cut Wood → Assemble → Paint → Install
 - C) Gather Materials → Design → Assemble → Cut Wood → Paint → Install
 - D) Install → Design → Paint → Assemble → Cut Wood → Gather Materials
-

Q60. Which principle states "Choosing the best solution involves evaluating different approaches and selecting the most efficient one"?

- A) Problem Understanding
 - B) Problem Simplification
 - C) Abstraction
 - D) Solution Selection and Design
-
-

Section B: Short Questions

Instructions: Answer each question in 2–5 sentences.

4.1 Computational Thinking

Q1. Define Computational Thinking.

Computational Thinking (CT) is a problem-solving process that involves a set of skills and techniques to solve complex problems in a way that can be executed by a computer. It is not limited to computer science and can be applied in everyday life, biology, mathematics, and other fields. Its key components include decomposition, pattern recognition, abstraction, and algorithms.

Q2. What is decomposition? Give one example.

Decomposition is the method of breaking down a complicated problem into smaller, more manageable components. For example, building a birdhouse can be decomposed into steps: designing, gathering materials, cutting wood, assembling, painting, and installing. This makes even complex tasks easier to handle one at a time.

Q3. What is pattern recognition in computational thinking?

Pattern recognition involves identifying and understanding regularities or patterns within a set of data or problems. For example, the area of squares follows a pattern where each area can be found by adding consecutive odd numbers (e.g., $1+3+5 = 9$ for side length 3). Recognizing such patterns helps create faster and more efficient solutions.

Q4. Define abstraction with an example.

Abstraction is the process of hiding complex details while exposing only the necessary parts, helping to reduce complexity. For example, making a cup of tea can be described at a high level as: boil water, add tea, wait, pour — without mentioning the complex chemistry or physics involved. It allows us to focus on the important aspects of a problem.

Q5. What is an algorithm? Give a real-life example.

An algorithm is a precise, step-by-step set of instructions followed to achieve a specific goal. A real-life example is a recipe for baking a cake, which includes steps like preheating the oven, mixing ingredients, and setting a timer. Algorithms are used everywhere — from cooking to navigation directions to computer programs.

4.2 Principles of Computational Thinking

Q6. What is meant by "Problem Understanding" in computational thinking?

Problem understanding involves identifying the core issue, defining the requirements, and setting the objectives before attempting a solution. It is the first and most important step in the problem-solving process. Without clearly understanding the problem, any solution designed may be ineffective or incorrect.

Q7. What is problem simplification? Give an example.

Problem simplification involves breaking down a large problem into smaller, more manageable sub-problems. For example, designing a website can be simplified by dividing it into sub-tasks: designing the layout, creating content, and coding the functionality. This approach makes large problems easier to solve step by step.

Q8. What is "Solution Selection and Design"?

Solution selection and design involves evaluating different approaches to solve a problem and choosing the most efficient one. After selecting the best approach, a detailed plan or algorithm is designed to implement the solution. This ensures the final solution is both effective and optimized.

4.3 Algorithm Design Methods

Q9. What are flowcharts and why are they important?

Flowcharts are visual representations of the steps in a process or system, depicted using different symbols connected by arrows. They are important because they provide clarity, aid communication, help in problem-solving by identifying bottlenecks, and serve as documentation. They are widely used in computer science, engineering, and business.

Q10. Name and describe any THREE flowchart symbols.

- **Oval (Terminal):** Represents the start or end of a process, labeled "Start" or "End."
- **Rectangle (Process):** Represents a process, task, or operation to be performed.
- **Diamond (Decision):** Represents a decision point where the flow branches based on a yes/no or true/false condition.

Q11. What is pseudocode and how does it differ from actual code?

Pseudocode is a method of representing an algorithm using simple, informal language that combines the structure of programming with the readability of plain English. Unlike actual code, pseudocode cannot be run on a computer — it is used to plan and explain the logic of an algorithm. It allows programmers to focus on problem-solving without worrying about programming language syntax.

Q12. List THREE benefits of using pseudocode.

1. **Clarity:** It helps understand the logic of the algorithm without worrying about syntax.
2. **Planning:** It allows programmers to outline their thoughts and plan algorithm steps before coding.
3. **Communication:** It is a universal way to convey algorithm steps, making it easier to discuss with team members who may use different programming languages.

Q13. Compare flowcharts and pseudocode. (Any two differences)

Feature	Flowchart	Pseudocode
Format	Uses graphical symbols and arrows	Uses structured plain language
Understanding	Visual — like watching a movie	Sequential — like reading a story
Best for	Seeing overall flow at a glance	Documenting logic for conversion to code

Q14. Write pseudocode to check if a number is even or odd.

```
Procedure CheckEvenOdd(number)
  Input: No.
  Output: "Even" or "Odd"
  Begin
    if (No. % 2 == 0) then
      print "No. is Even"
    else
      print "No. is Odd"
    End if
  End
```

Q15. Describe the flowchart for a login system as explained in the chapter.

The login system flowchart starts by asking the user to input a username and password. It then checks whether the credentials are valid. If correct, access is granted. If incorrect, the attempt counter increases. After 5 failed attempts, the account is locked and the user is alerted.

4.4 Evaluation Techniques

Q16. What is time complexity? Why is it important?

Time complexity measures how fast or slow an algorithm performs and shows how the running time changes as the size of the input increases. It is expressed using Big O notation such as $O(n)$, $O(\log n)$, or $O(n^2)$. It is important because it helps compare different algorithms to determine which one is faster and more efficient.

Q17. What is space complexity?

Space complexity measures the amount of memory an algorithm uses relative to the size of its input. It considers both the memory needed to store the input data and any additional memory used during execution. Understanding space complexity helps ensure an algorithm does not use more memory than available.

4.5 Dry Run and Simulation

Q18. What is a dry run? Why is it useful?

A dry run involves manually going through an algorithm or flowchart step-by-step with sample data to identify any errors before running it on a computer. It helps catch logical mistakes early in the development process, saving time and effort. Many professional programmers use dry running as a key debugging technique.

Q19. Perform a dry run of the "FindMax" pseudocode with inputs num1 = 10 and num2 = 15.

1. Input: num1 = 10, num2 = 15
 2. Check if num1 > num2 → 10 > 15 → **False**
 3. Since condition is False, execute else → max = num2 = **15**
 4. Output: **15**
-

Q20. What is simulation? Give two real-world examples.

Simulation is the use of computer programs to create a model of a real-world process or system to test ideas without needing real-life trials. Two examples are:

1. **Weather Forecasting:** Meteorologists input data like temperature and humidity into models to predict future weather.
 2. **Traffic Flow:** City planners simulate traffic to see how changes to roads or traffic lights affect the flow of cars.
-

Q21. List THREE benefits of simulation.

1. **Cost-Effective:** It is cheaper and faster to run simulations than real experiments.
 2. **Safe:** Dangerous situations like fires can be tested without real risk.
 3. **Repeatable:** The same simulation can be run multiple times with different settings to observe changes.
-

4.6 LARP

Q22. What is LARP and why is it important?

LARP stands for Logic of Algorithms for Resolution of Problems. It is a fun and interactive way to learn how algorithms work by running them and seeing the results. LARP is important because it helps students understand how algorithms process data, see the effect of different inputs on outputs, and practice writing and improving their own algorithms.

Q23. Write a LARP algorithm to check if a number is even or odd.

```
START
  WRITE "Enter a number"
  READ number
  IF number % 2 == 0 THEN
    WRITE "The number is even"
  ELSE
    WRITE "The number is odd"
  ENDIF
END
```

Q24. What commands does LARP use for input, output, and decisions?

- **WRITE** — to display messages or output to the user
- **READ** — to take input from the user
- **IF...THEN...ELSE...ENDIF** — for conditional decision-making
- **START** — to begin the algorithm
- **END** — to terminate the algorithm

4.7 Error Identification and Debugging

Q25. What are the three types of errors in algorithms? Define each briefly.

1. **Syntax Error:** Occurs when something is written incorrectly, such as missing a step or using the wrong symbol. It is the easiest to find.
 2. **Runtime Error:** Occurs during execution, such as attempting to divide by zero. The algorithm crashes or stops unexpectedly.
 3. **Logical Error:** Occurs when the algorithm's logic is wrong, causing incorrect results even though the algorithm runs without crashing. It is the hardest to detect.
-

Q26. Which type of error is hardest to find and why?

Logical errors are the hardest to find because the algorithm still runs successfully without any crash or error message, but produces incorrect results. Since no warning is displayed, the programmer must carefully trace the algorithm's logic to identify where the wrong condition or operation is being used.

Q27. What is debugging?

Debugging is the process of finding and fixing errors (bugs) in an algorithm, flowchart, or program. The term originates from an actual moth (bug) found causing problems in an early computer, which was then "debugged." Techniques like dry runs, reading error messages carefully, and testing with different inputs are used during debugging.

Q28. List the three common error messages in LARP and their meanings.

1. **Missing Step** — An important step was forgotten in the algorithm.
 2. **Undefined Variable** — A variable is being used before it has been defined.
 3. **Invalid Operation** — An operation not allowed is being performed, such as dividing by zero.
-

Q29. Why should error messages be read carefully during debugging?

Error messages often tell you exactly where the problem is in the algorithm, saving significant time during debugging. For example, in LARP, a "Missing Step" message immediately tells the programmer that a required step is absent. Carefully reading error messages allows programmers to locate and fix bugs quickly and efficiently.

Q30. What is the origin of the word "debugging"?

The term "debugging" comes from an actual bug — a moth — that was found causing problems inside an early computer. When the moth was physically removed from the machine, the process was called "debugging." Since then, the term has been used to describe the process of finding and fixing errors in programs and algorithms.

End of Question Bank — Chapter 4: Computational Thinking